

Proof Packets: A Derive-Don't-Trust Envelope for Accountable Agent Actions

Zain Dana Harper

Seattle, WA | [ORCID 0009-0001-7175-5393](https://orcid.org/0009-0001-7175-5393)

<https://github.com/HarperZ9>

July 2026

Abstract

Autonomous agents now take actions that matter. When they do, the record of what happened carries weight, and that record is usually a log the agent wrote about itself. A log can claim a success that never happened. This paper describes a small, deterministic verification envelope for a single agent action. We call it a proof packet. Its defining property is that the verdict is computed from checks over the packet's own materials, and the packet's stored claim of success is never taken as the verdict. A packet carries an objective and its scope, source and context references with content hashes, a routing and admission record, the action and a hash of its arguments, a side-effect class, a compensation reference, outputs with digests, and the artifact bodies themselves. A pure verifier runs a fixed set of named checks over this structure. It folds the results into one verdict from a three-value lattice: MATCH, DRIFT, and UNVERIFIABLE, where DRIFT dominates UNVERIFIABLE and UNVERIFIABLE dominates MATCH. No code path returns MATCH without a passing check set. A canned MATCH embedded in the packet is itself recorded as DRIFT. Missing evidence, such as an output digest with no embedded body or a required field that is absent, is reported as UNVERIFIABLE with the offending JSON path, and it never passes silently. Tamper that can be recomputed, such as a wrong artifact digest or an execution ordered before its admission, is DRIFT. The contribution is the envelope and its derive-don't-trust verdict. The domain content that any packet wraps stays outside the claim. We illustrate with fixtures across several domains, and we say plainly that each fixture is textbook material. For example, the "quantum error correction" fixture is a classical three-qubit repetition code. These illustrations show that the envelope generalizes across domains. None of them reports a domain result.

In plain terms

Suppose you hire someone to do one job, and when they are done they hand you a note that says "finished." You have to decide whether to act on that note. Maybe the job was to fix a pipe, and next you turn the water back on. A different job might be sending a payment, and now you are about to close the books on it. The note is the only thing you have, and the person who did the work is the same person who wrote it. Nothing about the note tells you whether it is true.

A proof packet replaces that note with a folder of evidence. The folder holds the parts a stranger could check: what the job was, what materials went in, what came out, and a record of when each step was allowed and when it ran. The key move is what you do with the folder. You run your own checks over the contents, and the answer comes from those checks. The "finished" label inside the folder does not set the answer.

The checks can return one of three answers. When a piece of evidence is missing, say the folder claims an output exists yet does not include the output itself, the honest answer is "cannot

tell.” We call that UNVERIFIABLE, and the report names which piece is missing. A second case is evidence that is present and contradicts the claim, say an output in the folder that does not match the fingerprint the folder recorded for it. There the answer is “wrong,” and we call it DRIFT. When every check passes, the answer becomes “confirmed,” which we call MATCH.

The design rules out one thing. The person who wrote the folder can stamp “finished” anywhere they like, and that stamp never wins. When the stamp sits over evidence that contradicts it, the checker still reports DRIFT. A stamp over a folder with a missing piece still yields UNVERIFIABLE. A folder can never vouch for itself. The answer is built only from the checks run over its contents.

The rest of the paper makes this precise. It pins down the shape of the folder, so any machine can produce and read one. The set of checks and the rule for folding them into one verdict are fixed too. The paper then shows the same folder shape wrapping claims from very different fields, and it states clearly that those examples show the folder works everywhere. The examples prove nothing new about any field.

1 Introduction

Plainly: an agent’s own log is a self-report, so the verdict comes from re-deriving the answer over the evidence.

An autonomous agent that reads a file, calls a tool, or writes to an external system produces, at best, a record of having done so. That record often becomes the basis for a later decision: deploy this, trust this result, pay this invoice. When it does, the integrity of the record matters as much as the action. The default record is a self-report. The agent logs its own success. A plain log does not separate a truthful “completed” from a fabricated one, and nothing in it forces the evidence into view that would let a third party re-derive the claim.

What we describe is a fixed-shape packet for one action, verified by a pure function. It adds no logging features, and a trusted execution environment is not required. The one design commitment is that the verdict is computed from checks over the packet’s own materials, so the packet cannot vouch for itself. This changes how trust flows. A verifier re-derives the status the materials support, and the status the producer wrote carries no weight on its own. Any disagreement between the derived status and the written one is itself a finding.

Contributions.

1. **A derive-don’t-trust verdict** (Section 3): a pure verifier that folds a fixed set of named checks into a three-value lattice. The verifier enforces one property in code, that an embedded verdict can never raise the result above what the checks support.
2. **An evidence-gap discipline** (Section 4): the verifier separates missing evidence, reported as UNVERIFIABLE and named by JSON path, from contradicted evidence, reported as DRIFT. It refuses to let a derivative failure, such as a hash mismatch, mask a more basic one, such as an absent field.
3. **A domain-agnostic envelope** (Section 2): the same packet shape wraps a claim in any domain, verified by re-execution of the check set. We demonstrate this across fixtures. Each fixture is an illustration and reports no domain result.

2 The packet

Plainly: a packet is one fixed-shape JSON record for one action, addressed by a hash so the same materials hash the same way anywhere.

A proof packet is a canonical JSON object for one action. Its required structure records the following. An **objective** holds the claim, its scope, and a success criterion. **source_refs** and **context_refs** each carry a content hash, so evidence can be pinned. A **route** holds the lane, the decider, a confidence, and a route reference. An **admission** record holds the decision, the policy, the authority, and an ordinal. The **action** holds identifiers, the tool, the kind, an arguments hash, an execution ordinal, and an idempotency key. A **side_effect** class carries a reversibility flag. A **compensation** block records how to undo the action. **outputs** each carry a name, a digest, and a reference. The **artifacts** are the bodies themselves, embedded as bytes. A **final_status** records the claimed outcome. The packet is content-addressed. A **packet_hash** is computed over the canonical form, with no clock or randomness inside the hash scope, so the same materials hash identically on any machine.

3 The verdict

Plainly: the verdict is a pure function of the checks, three values with a fixed precedence, and a claimed pass inside the packet can never lift it.

Verification is a pure function from a packet to a set of named check results and one derived verdict. The verdict lattice has three values. **MATCH** means every check passed. A confirmed contradiction gives **DRIFT**. **UNVERIFIABLE** means a check could not be completed. The fold uses a fixed precedence. If any check yields **DRIFT**, the verdict is **DRIFT**. Otherwise, if any check yields **UNVERIFIABLE**, the verdict is **UNVERIFIABLE**. Failing those, the verdict is **MATCH**. No code path returns **MATCH** without a passing check set. A packet may embed a claimed verdict. The verifier compares that claim to the derived verdict, and a disagreement is recorded as a failure that inherits the derived severity. An embedded **MATCH** over tampered materials stays **DRIFT**. Over an incomplete packet, that same embedded **MATCH** stays **UNVERIFIABLE**.

4 The checks

Plainly: a fixed suite of named checks reports the offending path on failure and keeps a missing piece separate from a contradicted piece. Each check fails toward caution.

The verifier runs a fixed suite. Each check is named, and its failures carry the offending JSON path. The design separates an evidence gap, reported as **UNVERIFIABLE**, from tamper, reported as **DRIFT**, wherever the gap can be detected before it perturbs the hash scope. A gap that also changes the hashed bytes is reported at the stricter **DRIFT** level, so it is never a silent pass either way.

- **Required fields, reference completeness.** Each required path must be present, and so must each field of every source, context, and output element. A missing one is **UNVERIFIABLE**, named by its path, and marks the packet structurally incomplete.
- **Artifact digests.** Each output digest is recomputed over its embedded artifact bytes. A wrong digest is **DRIFT**. A digest claimed with no embedded body is **UNVERIFIABLE** and never passes silently. Trusting a claimed digest on faith would let a phantom output reach **MATCH**.
- **Hash integrity, gated.** The packet hash is recomputed, and a mismatch is **DRIFT**. The check is gated on structural completeness, so that a missing required field, a missing source or context reference field, or a missing output element surfaces as its own **UNVERIFIABLE**. Without the gate, deleting such a field would trip only the derivative hash mismatch and mask the gap. A missing artifact body is the one gap the gate does not precede. It is reported as **DRIFT** via the recomputed hash, in addition to its **UNVERIFIABLE** digest gap. That is strictly more severe, so it is still never a silent pass.

- **Admission before execution.** The action’s admission record must match the action, and the execution ordinal must strictly follow the admission ordinal. An action executed before it was admitted is DRIFT.
- **Compensation for external writes.** An external-write side effect without a compensation reference is DRIFT.
- **Resolvable evidence, known states.** An output that cites a source reference not present in the packet is UNVERIFIABLE. An out-of-vocabulary state, side-effect class, or admission decision is DRIFT.

Each check is total, and the verifier fails closed. It produces a verdict for any input, and an input it cannot interpret degrades toward UNVERIFIABLE and never raises the verdict.

5 Illustrations, and what they do and do not show

Plainly: the fixtures show one thing, that any claim plus its evidence can be sealed and re-verified. Each fixture is textbook content and reports no research result.

The same envelope wraps a claim in any domain, and we ship fixtures that do so. There is a causal-inference fixture, an embodied sim-to-real fixture, and a quantum-error-correction fixture, among others. They illustrate one thing only: that a claim, together with its evidence and its negative controls, can be sealed into a packet and re-verified. We state the honest size of each. The domain content is textbook. The causal fixture recovers a backdoor adjustment set by brute-force subset enumeration on a small graph. The “quantum error correction” fixture is a classical three-qubit repetition code with a syndrome lookup. No quantum circuit is involved. The embodied fixture uses synthetic observations, and none come from real hardware. The fixtures also carry negative controls, which are deliberately broken variants that the envelope must reject. A passing fixture therefore shows that the verifier is not vacuous on that shape. None of the fixtures is a domain research result, and framing one as such would be the exact self-vouching the envelope exists to prevent. The contribution is that a single accountable-claim envelope, with a verdict the packet cannot forge, spans these domains unchanged.

6 Related work

Plainly: prior work gives integrity for stored records and re-checkable proofs, and our narrow addition is a verdict that the verifier derives at read time from the evidence.

Tamper-evident and Merkle-style logging [2] and content-provenance standards [3] give integrity for stored records. A proof packet applies the same content-addressing to a single agent action, and it adds a verdict that the verifier derives at read time from the evidence. Proof-carrying code [1] moves trust to a re-checkable artifact. A proof packet is a proof-carrying action record whose checks are structural and re-executable. Its checks are not a logical safety proof. Work on verifiable and constrained agents [4] shifts agent output toward the checkable. The packet is complementary. It sits at the after-the-fact record, and the pre-action constraint stays a separate concern. Reproducibility tooling for computational claims [5] shares the re-execution instinct. Our narrow addition is the derive-don’t-trust verdict and the explicit separation of gap from tamper.

7 Conclusion

Plainly: a record is worth only the checks a third party can re-run against it, so we make those checks the source of the verdict.

A record an agent writes about its own action is worth as much as the checks a third party can re-run against it. A proof packet makes those checks the source of the verdict, so the record cannot vouch for itself. Missing evidence is UNVERIFIABLE, and a contradiction yields DRIFT. MATCH has to be earned by a full pass of the checks. The envelope is small and domain-agnostic. It holds a claim in any domain to the size of its evidence.

References

- [1] Necula, G. C. *Proof-carrying code*. POPL, 1997.
- [2] Merkle, R. C. *A digital signature based on a conventional encryption function*. CRYPTO, 1987.
- [3] Coalition for Content Provenance and Authenticity. *C2PA Technical Specification*. <https://c2pa.org>, 2024.
- [4] Chen, B. *Making Failure Safe: A Constrained, Verifiable Agent Framework for Open-Web Data Collection*. arXiv:2607.00035, 2026.
- [5] Reproducible Builds Project. *Reproducible Builds: a set of software development practices*. <https://reproducible-builds.org>, 2026.